

Object-Oriented Programming in Java

Comprehensive Technical Report: Single Inheritance, Member Access, and Method Dispatch

1. PROBLEM STATEMENT / LABORATORY QUESTION

Problem Description:

Write a Java program to illustrate the concept of **Single Inheritance** using the `extends` keyword. Specifically:

- Create a superclass (base class) named `f` with an integer attribute `a` initialized to `10` and a member method `sum()` that computes and displays the result of `a + 34`.
- Create a subclass (derived class) named `s` that inherits from `f`, containing its own integer attribute `b` initialized to `54` and a member method `sub()` that prints the value of `b - 10`.
- In a driver class `Inherit1` containing the `main` method, instantiate the subclass `s` and demonstrate that its object can directly access both superclass and subclass state variables as well as invoke methods from both levels of the class hierarchy.

2. STEP-BY-STEP IMPLEMENTATION PROCESS

1. **Define the Superclass (f):** Declare class `f` with member variable `int a = 10` and method `void sum()`. This serves as the generalized parent class in the hierarchy.
2. **Derive the Subclass (s):** Use the `extends` keyword (`class s extends f`) to establish an "is-a" relationship. Define subclass-specific data member `int b = 54` and behavior `void sub()`.
3. **Define the Driver Class (Inherit1):** Provide the standard execution entry point `public static void main(String[] args)`.
4. **Subclass Instantiation:** Create an instance of the derived class: `s oc = new s();`. The default constructor chain allocates memory for both inherited members from `f` and new members of `s`.
5. **Access State Variables:** Access `oc.a` (inherited from `f`) and `oc.b` (defined in `s`) via dot notation to prove field accessibility.
6. **Invoke Instance Methods:** Execute `oc.sum()` (inherited behavior) followed by `oc.sub()` (subclass behavior).

3. EXECUTION ALGORITHM

Step	Action / Operation	Underlying Mechanism / Detail
Step 1	Start Execution	JVM loads classes <code>Inherit1</code> , <code>s</code> , and <code>f</code> into method area.
Step 2	Allocate Object <code>oc</code>	Instantiates class <code>s</code> ; invokes superclass constructor <code>f()</code> , initializes fields (<code>a = 10</code> , <code>b = 54</code>).

Step	Action / Operation	Underlying Mechanism / Detail
Step 3	Display Superclass Variable	Evaluates expression <code>oc . a</code> (resolves to 10) and prints string to standard output.
Step 4	Display Subclass Variable	Evaluates expression <code>oc . b</code> (resolves to 54) and prints string to standard output.
Step 5	Call <code>oc . sum ()</code>	Virtual method invocation pushes <code>sum ()</code> stack frame; computes $10 + 34 = 44$; prints result.
Step 6	Call <code>oc . sub ()</code>	Virtual method invocation pushes <code>sub ()</code> stack frame; computes $54 - 10 = 44$; prints result.
Step 7	Terminate	Pops main stack frame and exits JVM process with code 0.

4. UNIFIED MODELING LANGUAGE (UML) CLASS DIAGRAM

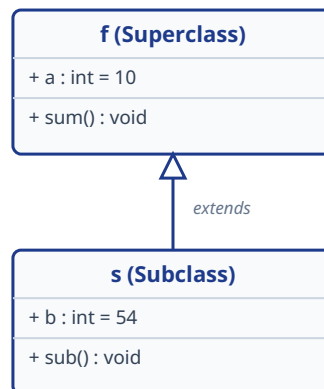


Figure: Standard UML Class representation of Single Inheritance between Superclass *f* and Subclass *s*.

5. SOURCE CODE (JAVA)

```
// Superclass Definition
class f {
    int a = 10;

    void sum() {
        System.out.println("sum is " + (a + 34));
    }
}

// Subclass inheriting from class f using the 'extends' keyword
class s extends f {
    int b = 54;

    void sub() {
        System.out.println("subtraction is " + (b - 10));
    }
}

// Driver Class with Main Entry Point
public class Inherit1 {
    public static void main(String[] args) {
        // Instantiate derived class object
        s oc = new s();

        // Access superclass and subclass member variables directly
        System.out.println("variable can access is " + (oc.a));
        System.out.println("variable can access " + (oc.b));

        // Invoke inherited method and subclass method
        oc.sum();
        oc.sub();
    }
}
```

6. SAMPLE PROGRAM OUTPUT

TERMINAL / STANDARD CONSOLE OUTPUT (JAVAC INHERIT1.JAVA && JAVA INHERIT1)

```
$ javac Inherit1.java $ java Inherit1 variable can access is 10 variable can access 54 sum
is 44 subtraction is 44
```

7. KEY OOP PRINCIPLES DEMONSTRATED

- **Code Reusability:** The subclass `s` automatically acquires field `a` and method `sum()` without duplicate declaration.
- **Single Inheritance:** Java implements single class inheritance where a child class extends exactly one direct parent class.
- **Object Memory Model:** An instance of `s` occupies a unified memory block on the heap containing slots for both `f.a` and `s.b`.